

PROJECTS AND EXAMS FOR THE MASTER COURSE

ARTIFICIAL INTELLIGENCE FOR SOFTWARE ENGINEERING

Academic year: 2026-2027

Teacher: Dr. Phuong Nguyen

Associate Professor, University of L'Aquila, Italy

Email: phuong.nguyen@univaq.it

Website: <https://www.disim.univaq.it/ThanhPhuong>

1. Introduction

- The project consists of an application realized in Python, running on Jupyter Lab, or the Colab platform, and a report to explain all the technical details. The project can be implemented by a group consisting of a minimum of 1 to a maximum of 5 students.
- We remind you that the project has a weight of 80% on the final evaluation. The remaining 20% of the assessment will consist of a single oral interview aimed at ascertaining the knowledge acquired during the course. In addition, there will also be a discussion of the project in the interview.
- The project must be delivered 10 days before the exam by sending an email to the teacher, expressing the willingness to take the exam. Therefore, the teacher and collaborators will consider the project developed by this deadline. This means that any subsequent changes will not be taken into account.

2. Common requirements

Each individual project chosen must be implemented with a good degree of flexibility and modularity and must take into account the following non-functional requirements/constraints:

- The data must be managed both in memory and through files present in the file system.
- The application must be designed in such a way that the use of one of the two persistence layers (memory or files) involves few changes to the application itself and, in particular, the representation (in terms of class / interfaces and relative methods) and the interaction with the so-called business logic must remain unchanged;
- Projects need to be regularly committed to GitHub. Before the exam, the lecturer will check each repository to see the history, and understand who has contributed what in the project.
- Write a document to explain the work.

IMPORTANT NOTES

The evaluation of the application will be conducted considering various aspects such as:

- Completeness of implementation.
- Use of conventions in writing Python code.
- Organization of the application in functions.
- Documentation through comments of the most significant portions of code.

Each project must be accompanied by a short report whose template will be found within the project associated with the GitHub repository.

It is highly advisable to use LaTeX to compose the report. This is a preparation step towards the master thesis, where students are expected to be capable of using LaTeX as well as the editing environment.

There are the following projects:

1. Project 1: Issue Report Classification

- Datasets: NLBSE'24 tool competition on issue report classification, 3,000 samples.
<https://nlbse2024.github.io/tools/>.
- Type: Multi-label classification.
- Implementing Machine Learning or Deep Learning classifiers.
- Evaluating the classifiers using suitable metrics.
- Splitting data for k-fold cross validation technique.
- Selecting a suitable machine learning technique.
- Running the evaluation.
- Comparing with the dedicated baselines (based on Sentence Transformers).
- Analyzing the obtained results.

2. Project 2: Code Comment Classification

- Datasets: NLBSE'23 tool competition on issue report classification, 6,738 samples.
<https://nlbse2023.github.io/tools/>.
- Type: Multi-label classification.
- Implementing Machine Learning or Deep Learning classifiers.
- Evaluating the classifiers using suitable metrics.
- Splitting data for k-fold cross validation technique.
- Selecting a suitable machine learning technique.
- Running the evaluation.
- Comparing with the dedicated baselines (Transformer-based RoBERTa).
- Analyzing the obtained results.

3. Project 3: Toxic Comment Classification

- Website: <https://www.kaggle.com/c/jigsaw-toxic-comment-classification-challenge>
- Datasets: Six categories (toxic, severe_toxic, obscene, threat, insult, identity_hate)
- Type: Multi-label classification.
- Implementing Machine Learning or Deep Learning classifiers.
- Evaluating the classifiers using suitable metrics.
- Splitting data for k-fold cross validation technique.
- Selecting a suitable machine learning technique.
- Running the evaluation.
- Comparing with the dedicated baselines.
- Analyzing the obtained results.

4. Project 4: Summarization of GitHub issues

- Dataset: <https://www.kaggle.com/datasets/davidshinn/github-issues>
 - Running the evaluation.
 - Analyzing the obtained results.
 - Generating textual descriptions for commits, issues.
 - Requirements: Summary a long text to yield a short/informative one.
 - Models: Transformer, BERT.
 - Text summarization: HuggingTransformer, T5, to name a few.
5. **Project 5:** Summarization of Stack Overflow Posts
- Dataset: <https://github.com/Bonankou/SOSum-A-Dataset-of-Extractive-Summaries-of-Stack-Overflow-Posts-and-labeling-tools>
 - Paper: [SOSum: A Dataset of Stack Overflow Post Summaries](#), [Automated Summarization of Stack Overflow Posts](#)
 - Running the evaluation.
 - Analyzing the obtained results.
 - Requirements: Summary a long text to yield a short/informative one.
 - Models: Transformers, BERT, LLMs (Llama 2), to name a few.
6. **Project 6:** Summarization of GitHub README.MD
- Dataset: <https://github.com/MDEGroup/GitSum>
 - Paper: [Too long, didn't read: Automatic summarization of GitHub README.MD with Transformers](#)
 - Running the evaluation.
 - Analyzing the obtained results.
 - Requirement: Summary a long text to yield a short/informative one.
 - Models: Transformer, BERT.
7. **Project 7:** Data leakage in LLMs for software engineering tasks
- Description: When being prompted, LLMs can reveal confidential information about the models.
 - Paper: [LessLeak-Bench: A First Investigation of Data Leakage in LLMs Across 83 Software Engineering Benchmarks](#)
 - Generative AI models: Llama, gpt-oss-20B, Qwen, Mistral.
8. **Project 8:** Memorization of public SE datasets in LLMs
- Papers: [Decoding Secret Memorization in Code LLMs Through Token-Level Characterization](#), [Do LLMs Memorize Recommendation Datasets? A Preliminary Study on MovieLens-1M](#).
 - Tasks: Investigating whether LLMs memorize some specific datasets used in Software Engineering, e.g., CodeSearchNet, Stack Overflow, CWE, CodeXGlue.
 - Generative AI models: Llama, gpt-oss-20B, Qwen, Mistral, to name a few.
9. **Project 9:** Hallucinations in LLMs with software engineering tasks.
- Tasks: Investigating different types of hallucinations in SE; Propose possible solutions to deal with hallucinations; Using machine unlearning to tackle hallucinations.

- References: [Machine Unlearning of Pre-trained Large Language Models](#).
10. **Project 10:** Bias and Fairness in LLMs for software engineering tasks:
 - Tasks: Studying how biases in LLM training data are reflected in generated code or design decisions, and developing strategies to ensure fair and equitable outcomes.
 - Papers: [Dealing with Popularity Bias in Recommender Systems for Third-party Libraries: How far Are We?](#); [Bake Two Cakes with One Oven: RL for Defusing Popularity Bias and Cold-start in Third-Party Library Recommendations](#).
 11. **Project 11:** Retrieval Augmented Generation (RAG): Concepts and Applications.
 - Tasks: Basic concepts in RAG; Using RAG to mitigate hallucinations.
 - References: [Detecting Malicious Source Code in PyPI Packages with LLMs: Does RAG Come in Handy?](#)
 - Generative AI models: Llama, gpt-oss-20B, Qwen, Mistral, to name a few.
 12. **Project 12:** Adversarial attacks to RAG-based recommender systems: Potential risks and countermeasures.
 - Tasks: Security of LLM-generated Code: Investigating how often LLMs introduce security vulnerabilities and developing techniques for LLMs to generate more secure code and perform better vulnerability detection. Dealing with attacks: Using LLMs via In-Context Learning; Prompt Engineering and Chain-of-Thought.
 - References: [Poison-RAG: Adversarial Data Poisoning Attacks on Retrieval-Augmented Generation in Recommender Systems](#)
 - Generative AI models: Llama, gpt-oss-20B, Qwen, Mistral, to name a few.
 13. **Project 13:** LLM-based Multi-Agent Systems (MAS) for Software Engineering:
 - Tasks: MAS for different Software Engineering tasks: Code generation, Debugging, Testing, Summarization; Different issues with MAS: Security (Dealing with adversarial attacks), Optimization, Prompt Engineering.
 - References: [Agents](#); Frameworks: LangChain, LangGraph; [Teamwork makes the dream work: LLMs-Based Agents for GitHub README.MD Summarization](#); [LLM-Based Multi-Agent Systems for Software Engineering: Literature Review, Vision and the Road Ahead](#)
 14. **Project 14:** Evaluation and Benchmarking of LLM-based Multi-Agent Systems for Software Engineering
 - Tasks: Creating standardized and challenging benchmarks and evaluation metrics to accurately assess the performance, correctness, and reasoning capabilities of LLMs in software engineering contexts.
 - Generative AI models: Llama, gpt-oss-20B, Qwen, Mistral, to name a few.
 - Analyzing the obtained results.